Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Smart Courier Delivery and Route Optimization System

Team Name: *Team #*	*Route Rangers* *DSCPP-III-2026-T332* *Section: J and ML4*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Rai, Arpit – 2510012741* *arpitrai012@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Vardhan, Shreshth – 2510379364* *vardhanshreshth@gmail.com* 
Mentor:	*Ms. Isha Deshmukh*

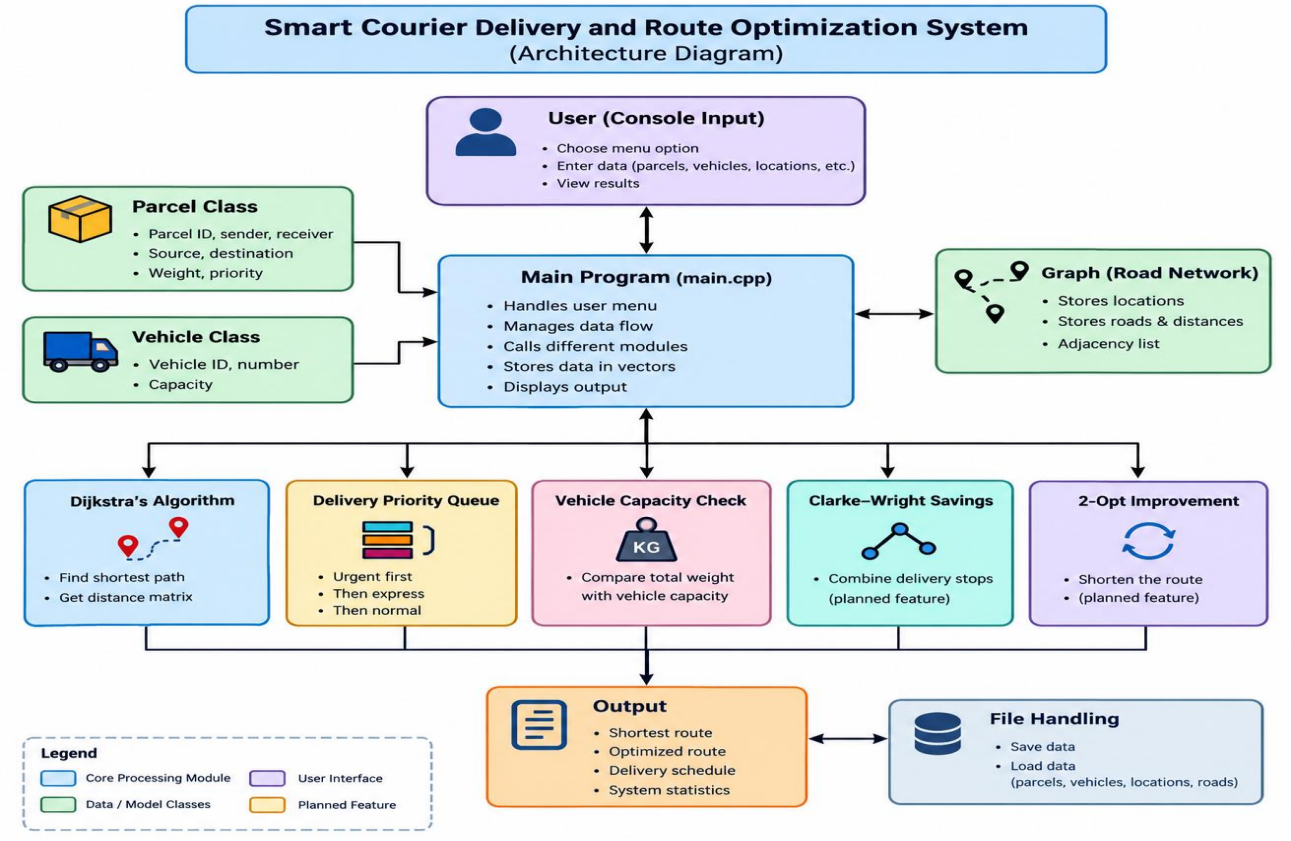
PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The Smart Courier Delivery and Route Optimization System is a C++ console application planned to help organize parcel deliveries more efficiently. The user can register parcels and vehicles, record parcel weight and delivery priority, and create a small road network by adding locations and distances between them. The system will use data structures and algorithms to find useful routes and organize deliveries. A weighted graph represented with an adjacency list will model the road network, while Dijkstra's algorithm will find the shortest path between two locations. A priority queue will help process urgent parcels before express and normal parcels. Vehicle-capacity checks will prevent assigning more parcel weight than a vehicle can carry. For a more advanced route plan, the project aims to include a Clarke–Wright savings method for combining delivery stops and a 2-Opt improvement step to reduce route distance. The project is intended as an educational simulation using sample locations and distances, not a live navigation service.

Updated Project Approach and Architecture

The project is being developed in C++ as a menu-driven console application in Visual Studio Code, using a GCC/MinGW compiler. It does not require a network connection or communication protocol because all inputs are entered locally by the user. The program is organized around Parcel and Vehicle classes. These classes keep related information together and demonstrate object-oriented programming concepts such as encapsulation, constructors and member functions. Vectors from the C++ Standard Library store collections of parcels and vehicles. The planned Graph module will store locations and weighted roads in an adjacency list. Dijkstra's algorithm will use a priority queue to calculate a shortest path. A separate delivery-priority queue will rank parcels by urgency. Vehicle capacity validation will compare the total parcel weight with the selected vehicle's capacity. The route-planning stage is planned to use Clarke–Wright savings to suggest combined routes and 2-Opt to try to improve a route by changing the order of stops. File streams will be used to save and load project data. The implementation will be developed in small modules and tested after each feature is connected.



Tasks Completed

Task Completed	Team Member
Selected the courier delivery and route optimization problem and defined its main purpose.	Arpit Rai and Shreshth Vardhan
Designed the initial Parcel class with parcel ID, sender, receiver, source, destination, weight and priority fields.	Arpit Rai
Designed the initial Vehicle class with vehicle ID, vehicle number and carrying capacity.	Shreshth Vardhan
Created a basic menu plan for adding and viewing parcels and vehicles, using vectors to store records.	Both members

Challenges/Roadblocks

One challenge is connecting the different algorithms into one program without making the code difficult to manage. The basic parcel and vehicle classes are a starting point, but the graph, shortest-path calculation and route-improvement stages still need to be integrated and tested. Another challenge is making sure the route calculation uses valid location IDs and that the system handles cases where no route exists. The program must also reject invalid weights, priorities and vehicle capacities. During the first compilation attempts, a linker/build error was encountered, so the build process needs to be checked before adding more modules. We will solve these issues by compiling after each change, testing with a small road network first, and checking both normal and invalid inputs. The route optimization algorithms will be tested with hand-calculated examples so we can compare their output and explain why a route was selected. We will also keep the code split into clear classes and header files.

Tasks Pending

Task Pending	Team Member (to complete the task)
Fix and verify the build; add input validation and support for names with spaces.	Arpit Rai
Implement the weighted road graph and Dijkstra shortest-path algorithm.	Shreshth Vardhan
Implement priority-based delivery processing and vehicle-capacity checks.	Both members
Add Clarke–Wright savings and 2-Opt route improvement, then connect them to the menu.	Both members
Add file saving/loading, integrated tests, screenshots and final documentation.	Both members

Project Outcome/Deliverables

The planned deliverable is a working C++ console application for a small courier delivery scenario. It will allow users to add and view parcels and vehicles, represent roads between locations, calculate shortest routes, prioritize urgent deliveries and check vehicle load capacity. The advanced route-planning stage will compare possible delivery combinations using the Clarke–Wright savings method and attempt to improve stop order with 2-Opt. The project will also include sample input data, saved project records, source code, test cases and a short user guide. The final demonstration will show how vectors, queues, priority queues, graphs, searching/sorting where applicable, file handling, classes and constructors are used to solve a practical problem.

Progress Overview

The project is currently at the early implementation stage. The topic and overall design have been selected, and the first version of the Parcel and Vehicle classes and a basic menu have been drafted. These parts form the foundation, but the complete application is not yet finished or fully tested. The road graph, Dijkstra shortest-path feature, delivery-priority queue, vehicle-capacity assignment, route construction, 2-Opt improvement and file persistence remain pending or in progress. The initial build also needs to be verified after a linker error occurred during an early run. No feature is being marked as fully complete until it compiles and passes a repeatable test. The next priority is to get the basic program building correctly, then add and test each algorithm one at a time. Advanced route optimization and final documentation may take the most time, so the team will divide the modules and integrate them gradually.

Codebase Information

The basic parcel and vehicle classes and menu structure have been drafted. Compilation and testing are yet to be verified. Graph-based routing, delivery prioritization, vehicle-capacity assignment, and route optimization are planned for later development.

Data Structures Used:

- **Vector:** Stores parcel and vehicle records.
- **Graph (planned):** Represents delivery locations and roads with distances.
- **Priority Queue (planned):** Helps process urgent deliveries before normal deliveries.

Algorithms Planned:

- **Dijkstra's Algorithm:** Finds the shortest path between two delivery locations.
- **Clarke–Wright Savings Algorithm:** Helps plan efficient routes for multiple deliveries.
- **2-Opt Algorithm:** Improves a delivery route by reducing its total distance.

Object-Oriented Programming:
The project uses classes, objects, constructors, encapsulation, and member functions to organize parcel and vehicle information.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Source review	Pass	Parcel and Vehicle class structure reviewed.
Compilation/build	Pass	Compilation complete.
Add/view parcel	Pass	Program builds successfully.
Add/view vehicle	Pass	Program builds successfully.
Graph and route algorithms	In progress	Implementation using a small network with known distances to be done.

Deliverables Progress

Project design and class outline. Substantially complete

Parcel and vehicle menu. Completed

Weighted graph and Dijkstra. Partly planned; integration pending

Priority delivery and capacity validation. Substantially complete

Clarke-Wright and 2-Opt route optimization. Partly planned; integration pending

File saving/loading. In progress

Testing, screenshots and user guide. In progress